Probeklausur: Mikrocomputertechnik 3

Wrap-Up & Prüfungsvorbereitung · MCT3

Musterlösung

Deckblatt

Name	
Matrikelnummer	
Kurs / Studiengruppe	

Hinweise zur Bearbeitung

- **Dauer:** 90 Minuten
- **Erlaubte Hilfsmittel:** Nicht-programmierbarer Taschenrechner (keine Skripte, kein Smartphone)
- **Maximal erreichbare Punktzahl:** 100 Punkte

Punkteübersicht

Aufgabe	Thema	Punkte
1	Speicherphysik, Hierarchie und Caches	20
2	Interrupts, Timer, PWM	25
3	DMA, CPU-Architekturen, Branch Prediction	30
4	Multicore, Busse und Edge AI	25
	Summe	**100**

Aufgabe 1: Speicherphysik, Hierarchie und Caches (20 Punkte)

1.1 SRAM vs. DRAM (5 Punkte)

Nennen Sie je zwei charakteristische Unterschiede zwischen SRAM und DRAM (Zellaufbau, Refresh, Dichte/Flüchtigkeit).

Musterlösung 1.1

- **Zellaufbau:** SRAM typisch 6T-Flip-Flop; DRAM 1T1C (Transistor + Kondensator).

- **Refresh:** DRAM braucht periodischen Refresh (Ladung sickert ab); SRAM hält den Zustand, solange Versorgungsspannung anliegt.
- **Dichte/Kosten:** DRAM dichter/günstiger pro Bit; SRAM schneller, aber flächenintensiver. Beide volatil.

1.2 AMAT-Berechnung (6 Punkte)

Ein SoC verfügt über einen L1-Cache und langsamen DRAM.

- Hit Time L1: 2 Taktzyklen
- Miss Penalty (DRAM): 150 Taktzyklen
- Hit Rate: 96 %

Berechnen Sie die Average Memory Access Time (AMAT) in Taktzyklen. Formel und Rechenweg angeben.

Musterlösung 1.2

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

$$\text{Miss Rate} = 1 - 0{,}96 = 0{,}04$$

$$\text{AMAT} = 2 + (0{,}04 \times 150) = 2 + 6 = 8 \text{ Taktzyklen}$$

1.3 Cache-Mapping (4 Punkte)

Erklären Sie *Thrashing* bei einem Direct-Mapped-Cache. Wie löst ein 4-Way Set-Associative Cache das Problem hardwareseitig?

Musterlösung 1.3

Thrashing: Beim Direct Mapped hat jeder RAM-Block genau einen festen Cache-Platz (Index). Wechseln zwei Adressen mit demselben Index ab, verdrängen sie sich ständig → Miss Rate nahe 100 %, obwohl der Rest des Caches leer sein kann.

4-Way: Derselbe Index führt auf ein *Set* mit 4 Ways. Beide Blöcke können im Set nebeneinander liegen, ohne sich zu verdrängen.

1.4 Hardwarenahes C und Lokalität (5 Punkte)

```
for (int col = 0; col < 1000; col++) {
    for (int row = 0; row < 1000; row++) {
        bild[row][col] = 255 - bild[row][col];
    }
}
```

Warum ist die Performance mit Caches schlecht? Wie muss der Code umgeschrieben werden?

Musterlösung 1.4

C speichert 2D-Arrays *Row-Major*. Der Code läuft *Column-Major* → große Sprünge, Cache-Lines werden ungenutzt verdrängt (viele Misses).

Lösung: Schleifen tauschen — äußere `row`, innere `col` (sequenzieller Zugriff, hohe räumliche Lokalität).

Aufgabe 2: Interrupts, Timer und Aktorik (25 Punkte)

2.1 Interrupt-Controller (8 Punkte)

Nennen Sie für CLINT und PLIC je ein typisches Einsatzbeispiel und begründen Sie kurz die hardwareseitige Trennung.

Musterlösung 2.1

- **CLINT:** lokaler Timer (`mtime`/`mtimecmp`), Software-/IPI — kernnah, geringe Latenz für OS-kritische Events.
- **PLIC:** externe Peripherie (UART, I2C, GPIO, …) — bündelt, priorisiert und routet viele Quellen.

Trennung: kritische lokale Events ohne Warteschlange hinter hunderten Peripherie-IRQs.

2.2 Hardware-Timer und PWM (10 Punkte)

Systemtakt 50 MHz, `PSC = 49`, `ARR = 10000`.

1. Berechnen Sie die PWM-Frequenz in Hz (vereinfachtes Modell $f_{\text{pwm}} = f_{\text{cnt}}/\text{ARR}$).
2. Welchen `CCR`-Wert brauchen Sie für 60 % Duty Cycle?

Musterlösung 2.2

1. $$f_{\text{cnt}} = \frac{50\,\text{MHz}}{\text{PSC}+1} = \frac{50\times10^6}{50} = 1\,\text{MHz}$$
$$f_{\text{pwm}} = \frac{f_{\text{cnt}}}{\text{ARR}} = \frac{10^6}{10000} = 100\,\text{Hz}$$
2. CCR $= 10000 \times 0{,}60 = 6000$

2.3 Nested Interrupts (7 Punkte)

Was ist das *Clobbering* von `mepc`, wenn in einer ISR (nach erneut gesetztem `MIE=1`) ein weiterer Interrupt eintrifft? Welche Maßnahme ist nötig, bevor `MIE` in der ISR wieder auf 1 gesetzt wird?

Musterlösung 2.3

Trifft nach `MIE=1` ein weiterer Interrupt in der ISR ein, überschreibt die Hardware `mepc` mit dem PC *innerhalb* der äußeren ISR — die ursprüngliche Rücksprungadresse zum Main-Programm geht verloren (*Clobbering*).

Vor erneutem `MIE=1` müssen `mepc` (und typisch `mstatus`) auf dem Stack gesichert werden.

Aufgabe 3: DMA, CPU-Architekturen und Branch Prediction (30 Punkte)

3.1 Direct Memory Access (8 Punkte)

DMA überträgt 10000 Bytes aus einem SRAM-Array an UART-TX.

1. Inkrement-Modi für `SRC_ADDR` und `DST_ADDR` (An/Aus)? Begründung.

2. Hauptvorteil gegenüber interrupt-basiertem Senden (Interrupt Storm)?

Musterlösung 3.1

1. **SRC:** Inkrement an (Array durchlaufen). **DST:** Inkrement aus (immer dasselbe TX-Register; sonst Bus Fault / falsche Adressen).

2. Kein ISR-Overhead pro Byte — DMA als Bus-Master kopiert SRAM → UART, CPU-Last nahe 0 %.

3.2 Cache-Kohärenz bei DMA (5 Punkte)

L1-D-Cache mit Write-Back: Warum kann DMA *Stale Data* senden? Software-Abhilfe?

Musterlösung 3.2

CPU schreibt nur in den Cache; RAM bleibt veraltet. DMA liest den RAM und sendet alte Bytes.

Abhilfe: vor DMA-Start *Cache Clean/Flush* des Puffers — oder Puffer in *uncached* Region (MPU/PMP/Linker).

3.3 Moderne CPU-Architekturen (7 Punkte)

```
1: add t0, t1, t2
2: sub t0, t4, t5
```

RAW oder WAR/WAW? Wie löst Register Renaming das Problem?

Musterlösung 3.3

WAW (beide schreiben `t0`; Befehl 2 braucht das Ergebnis von 1 nicht).

Renaming: Schreibziel von 1 → physisches Register z. B. `P10`, von 2 → `P11`. Ziele getrennt → beide parallel ausführbar.

3.4 2-Bit-Branch-Predictor (10 Punkte)

Ein 2-Bit saturierender Zähler startet in `11` (*Strongly Taken*). Kodierung: `00` Strongly NT, `01` Weakly NT, `10` Weakly T, `11` Strongly T. Vorhersage: `1x` → Taken, `0x` → Not Taken.

Sequenz der tatsächlichen Sprünge: **T, T, NT, NT, T**.

1. Geben Sie für jeden Schritt Startzustand, Vorhersage, Ist-Ergebnis und Folgezustand an.
2. Wie viele Treffer?

Musterlösung 3.4

Schritt	Start	Vorhersage	Ist	Folge
1	`11`	T	T	`11`
2	`11`	T	T	`11`
3	`11`	T	NT	`10`
4	`10`	T	NT	`01`
5	`01`	NT	T	`10`

Treffer: 2 von 5 (Schritte 1 und 2).

Aufgabe 4: Multicore, Busse und Edge AI (25 Punkte)

4.1 MESI-Protokoll (6 Punkte)

Beide Kerne haben Variable `x` in **S**. Core 1 führt `x = 42;` aus. Welche Bus-Nachricht? Zustände danach in Core 0 und Core 1?

Musterlösung 4.1

Core 1 sendet **Invalidate**-Broadcast. Core 0 → **I**, Core 1 nach dem Write → **M**.

4.2 Hardware-Atomics (5 Punkte)

Warum führt `while(lock == 1); lock = 1;` zu einer Race Condition? Welche zwei RISC-V-A-Extension-Befehle lösen das?

Musterlösung 4.2

Lesen und Schreiben sind getrennte Befehle: beide Kerne können „frei“ sehen und den Lock setzen → kritischer Bereich doppelt betreten.

Lösung: `lr.w` (Load-Reserved) und `sc.w` (Store-Conditional).

4.3 Serielle Busse (4 Punkte)

Warum ist I2C gegen Kurzschlüsse resilienter als SPI Push-Pull, wenn zwei Geräte gleichzeitig treiben — und welcher Geschwindigkeitsnachteil folgt daraus?

Musterlösung 4.3

I2C: Open-Drain — nur aktiv LOW oder hochohmig; HIGH über Pull-ups. Konflikt LOW/„HIGH“ → Strom über Pull-up nach GND, kein Push-Pull-Kurzschluss.

Nachteil: langsame Rising Edges → typisch deutlich niedriger als SPI (z. B. 100–400 kHz vs. MHz).

4.4 KI-Hardware / Systolic Array (5 Punkte)

Warum ist ein Systolic Array (NPU) für neuronale Netze energieeffizienter als eine hochgetaktete General-Purpose-CPU?

Musterlösung 4.4

Milliarden identischer MACs. Die CPU leidet unter Memory Wall und Instruction-Decode-Overhead. Systolic Arrays reichen Daten in Hardware von Zelle zu Zelle weiter (Wiederverwendung) → weniger RAM-Zugriffe und weniger Front-End-Energie → höhere Effizienz (TOPS/Watt).

4.5 Quantisierung und Memory Wall (5 Punkte)

Ein Modell habe 250 000 Gewichte.

1. Vergleichen Sie den Speicherbedarf in Float32 vs. Int8.

2. Warum mildert Int8-Quantisierung die *Memory Wall* auf MCUs?

Musterlösung 4.5

1. Float32: $250\,000 \times 4\,\mathrm{B} = 1\,\mathrm{MB}$. Int8: $250\,000 \times 1\,\mathrm{B} = 250\,\mathrm{KB}$ (Faktor 4 kleiner).
2. Weniger Bytes → weniger DRAM-/Flash-Bandbreite und kürzere Ladezeiten; Integer-MACs oft günstiger als Float ohne FPU. Die ALU wartet weniger auf Speicher → Memory Wall wird entschärft.